



İST266 - Hipotez Testleri - ÖDEV

Hazırlayan

Hasan Efe Kocasü

2240329066

Ders Sorumluları

Doç. Dr. Yasemin Kayhan Atılğan

Arş. Gör. Dolunay Ezgi Seyhan

ÖDEV 1

Bölüm 1 – Kitle Parametrelerinin Hesaplanması

Hesaplanan Kitle Ortalaması $\mu = 50.10651655016294 \approx 50$

Hesaplanan Kitle Varyansı $\sigma^2 = 99.12312792831453 \approx 100$

Bölüm 2 – Farklı Örneklem Büyüklükleri ile Nokta Tahmini

$n = 10, n = 30, n = 100, n = 500$

Büyüklüklerinde örneklemir çekilmiştir. X'in dağılımının parametrelerine ait nokta tahmini için ise örneklem ortalaması $\bar{x}$ ve örneklem varyansı S^2 istatistikleri kullanılmıştır.

Elde edilen değerler;

n = 10 örneklemi için; $\bar{x} = 51.23863490214186$ $S^2 = 65.14485914208004$	n = 30 örneklemi için; $\bar{x} = 49.768142057628886$ $S^2 = 48.99186399951632$
n = 100 örneklemi için; $\bar{x} = 50.045342377970684$ $S^2 = 91.00381011345227$	n = 500 örneklemi için; $\bar{x} = 50.80455027187787$ $S^2 = 105.47933129173595$

Kitle parametreleri ise ;

$$\mu = 50.10651655016294 \approx 50$$
$$\sigma^2 = 99.12312792831453 \approx 100$$

Karşılaştıracak olursak $\bar{x}$ değerleri kitle ortalaması olan μ değerine oldukça yakın S^2 değerleri ise örneklem büyüklüğü büyüdükçe σ^2 değerine yaklaşmıştır. Yorumunu yapabiliriz.

Bölüm 3 – Yansızlık ve Tutarlılık Simülasyonu

Kitleden çekilen örneklerde her n değeri için **1000** adet örneklem ortalaması ve örneklem varyansı hesaplanmıştır.

Elde edilen değerler aşağıdaki gibidir;

$n = 10$ için

Örneklem Ortalaması : 50.033594742042666

Örneklem Varyansı : 9.98346703401207

$n = 30$ için

Örneklem Ortalaması : 50.077417967920056

Örneklem Varyansı : 3.3405473241528965

$n = 100$ için

Örneklem Ortalaması : 50.11983083020773

Örneklem Varyansı : 0.9733018129139838

$n = 500$ için

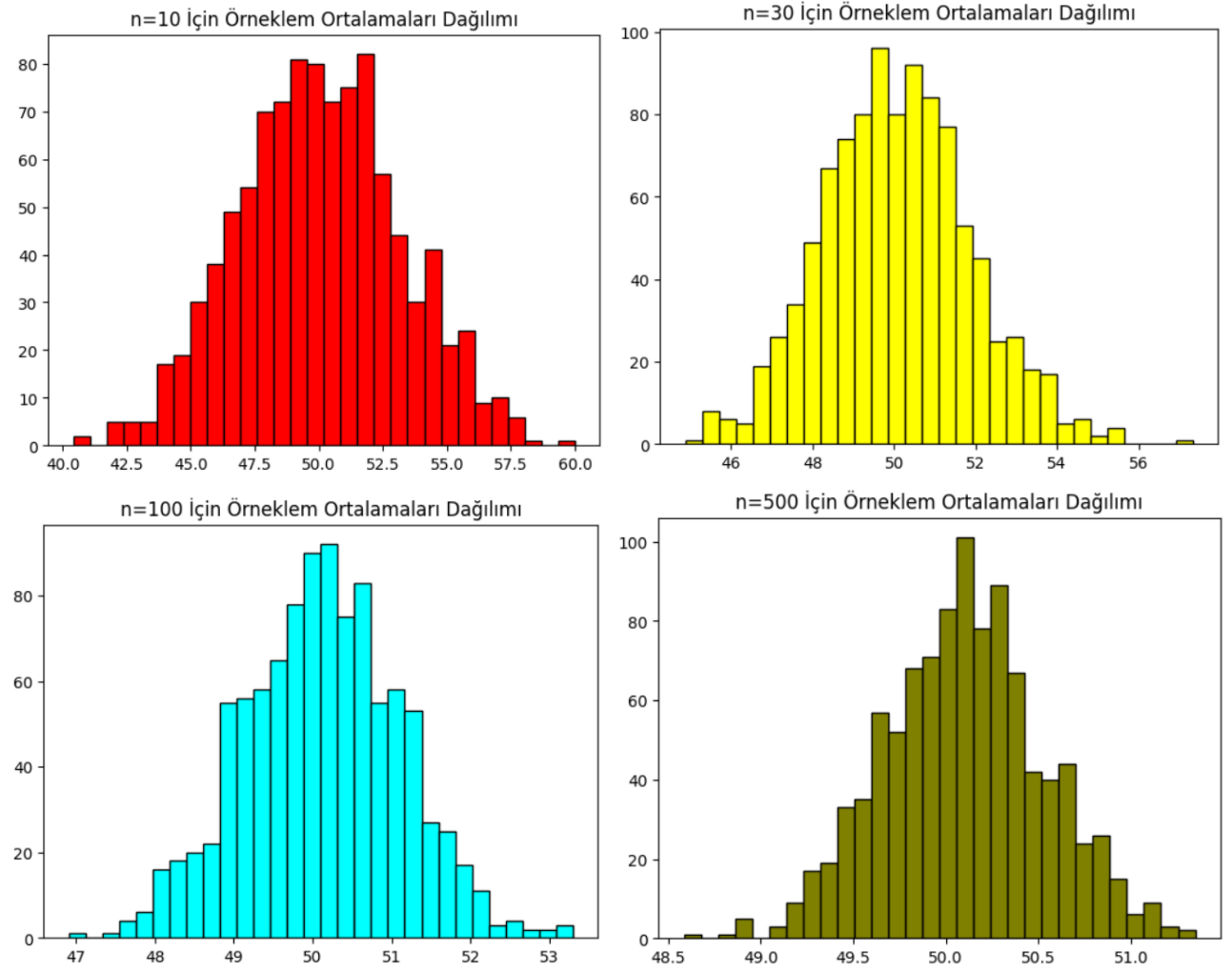
Örneklem Ortalaması : 50.099973242594444

Örneklem Varyansı : 0.18687106718520716

Yansızlık kavramını test etmek için tahmin edicinin beklenen değerinin tahmin etmek istediğimiz parametre değerine eşit çıkması beklenir, örneklem ortalaması $\bar{x}$ kitle ortalamasına μ eşit çıkmıştır bu sonuç bize örneklem ortalamasının $\bar{x}$ kitle ortalaması μ için yansız bir tahmin edici olduğunu gösterir.

Tutarlılık kavramını test edebilmek için örneklem büyüklüğü miktarı arttıkça tahmin etmek istediğimiz parametreye yakınsayıp yakınsamadığına bakarız. Elde ettiğimiz simülasyon sonuçlarından ise örneklem büyüklüğü n arttıkça tahmin etmek istediğimiz parametreye yakınsadığı, varyansının 0 olduğunu görüyoruz bu sonuç bize test ettiğimiz tahmin edicilerin tutarlılığı olduğunu göstermiştir. n (örneklem büyüklüğümüz) büyüdükçe kitle parametresi olan kitle ortalamasına μ yakınsadığını görüyoruz.

Örneklem büyüklüğü arttıkça tahminler parametre etrafına yoğunlaştığını histogram grafikleri yardımıyla görmüştür.



Bölüm 4 – Sonuç ve Değerlendirme

Örneklem büyüklüğümüz arttıkça örneklem parametrelerimiz kitle parametrelerine yaklaşmaktadır bu tutarlılık ilkesince tahmin edicinin (örneklem ortlaması $\bar{x}$) tutarlı olduğunu gösterir.

Yansızlık ise tahmin edicini tahmin etmek istediğimiz parametreye bağlı olmaksızın bize o kitle için parametre hakkında bilgiler vermesine yarayan kavramdır. Bu simülasyon sonucunda da örneklem ortalamasının yansız ve tutarlı bir tahmin edici olduğunu görmüş olduk.

ÖDEV 2

Tahmin Edicilerin Performansı

Kitle;

$$X \sim N(20, 15^2)$$

Kitle ortalaması:

$$\mu = 20$$

μ 'nün bilinmediği varsayımı altında üç farklı tahmin edici tanımlanmıştır:

$$T_1 = \bar{X} \quad T_2 = \frac{X_1 + X_2}{2} \quad T_3 = \frac{X_1 - 2X_2 + 3X_3}{6}$$

Ölçümler:

Tahmin edici	Yansızlık	Varyans	Hata Kareler Ortalaması
T ₁	0.001714921444847306	7.309909836875564	7.3099127778311255
T ₂	0.342679087543754	112.96634767182057	113.08377662886039
T ₃	14.205737200259803	95.33227380212516	297.1352432049704

Kitle varyansı:

$$\sigma^2 = 15^2 = 225$$

σ^2 'nin bilinmediği varsayımı altında iki farklı tahmin edici tanımlanmıştır:

$$T_1 = \frac{\sum (X_i - \bar{X})^2}{n-1} \quad T_2 = \frac{(X_1 - X_2)^2}{2}$$

Ölçümler:

Tahmin edici	Yansızlık	Varyans	Hata Kareler Ortalaması
T ₁	0.001714921444847306	7.309909836875564	9610101.650896564
T ₂	0.342679087543754	112.96634767182057	10200920695.041449

Bölüm 3 - Etkinlik

$$Efficiency = \frac{V(T_1)}{V(T_2)}$$

Kitle ortalaması tahmin edicilerinin ‘Etkinlik’ değerlerini yorumlayacak olursak;

T_2 ’nin T_1 tahmin edicisine göre etkinliği : 0.0647087383767743

T_3 tahmin edicisinin etkinliği hesaplanamaz çünkü etkinlik kavramını hesaplayabilmek için karşılaştırılan her iki tahmin edicinin de tutarlı ve yansız olması beklenir. T_1 tahmin edicisi de T_2 gibi tutarlı ve yansızdır anca varyans değerinin daha küçük olması onu daha ‘Etkin’ kılar bu da T_2 ’ye göre daha tercih edilebilir bir tahmin edici yapar.

Kitle varyansı tahmin edicilerinin ‘Etkinlik’ değerlerini yorumlayacak olursak;

T_2 ’nin T_1 tahmin edicisine göre etkinliği : 0.032842771027190194

T_1 tahmin edicisi de T_2 gibi tutarlı ve yansızdır ancak varyans değerinin daha küçük olması onu daha ‘Etkin’ kılar bu da T_2 ’ye göre daha tercih edilebilir bir tahmin edici yapar.

Bölüm 4 – Yorum ve Değerlendirme

Kitle ortalaması için; T_1 tahmin edicisi en iyi seçenek olacaktır diğer tahmin edicilere kıyasla. Çünkü T_3 tahmin edicisi yansız değildir bu yüzden iyi bir tahmin edici değildir. T_2 ile T_1 her ikisi de tutarlı ve yansızdır bunlar arasında en iyi tahmin ediciyi seçmek içinse etkinlik ölçümünü kullanarak inceleyebiliriz ve ölçüm sonucunda ise T_1 tahmin edicisi T_2 tahmin edicisinden varyansı daha küçük olması sebebiyle daha etkin çıkmıştır. Bu yüzden en iyi tahmin edici T_1 ’dir diyebiliriz.

Kitle varyansı için; T_1 tahmin edicisi en iyi seçenektir çünkü T_2 tahmin edicisine göre varyansı daha küçük olduğundan etkindir diyebiliriz. Bu yüzden en iyi tahmin edici de bu iki tahmin edici arasından T_1 ’dir.

Daha düşük hata kareler ortalaması demek; ortalamanın hata üretimi miktarına bakarak ne kadar gerçek parametreden saptığı ne kadarlık bir fark biriktirdiğini bize matematiksel olarak verir. Tahmin edici ile elde edilen değerler sonucunda varyans miktarı ne kadar parametrenin varyans miktarına eşit olursa o kadar hata kareler ortalamasının düşük olduğu da bu açıklamayı desteklemiştir.

ÖDEV 3

Kitle dağılımı;

$$X \sim U(0, \theta)$$

θ parametresi için aşağıdaki tahmin edici tanımlanmıştır:

$$\hat{\theta} = X_n$$

X_n deęerinin, verilen sıralı bilgisinden en büyük sıralı istatistik olduęu bilgisine erişebiliriz

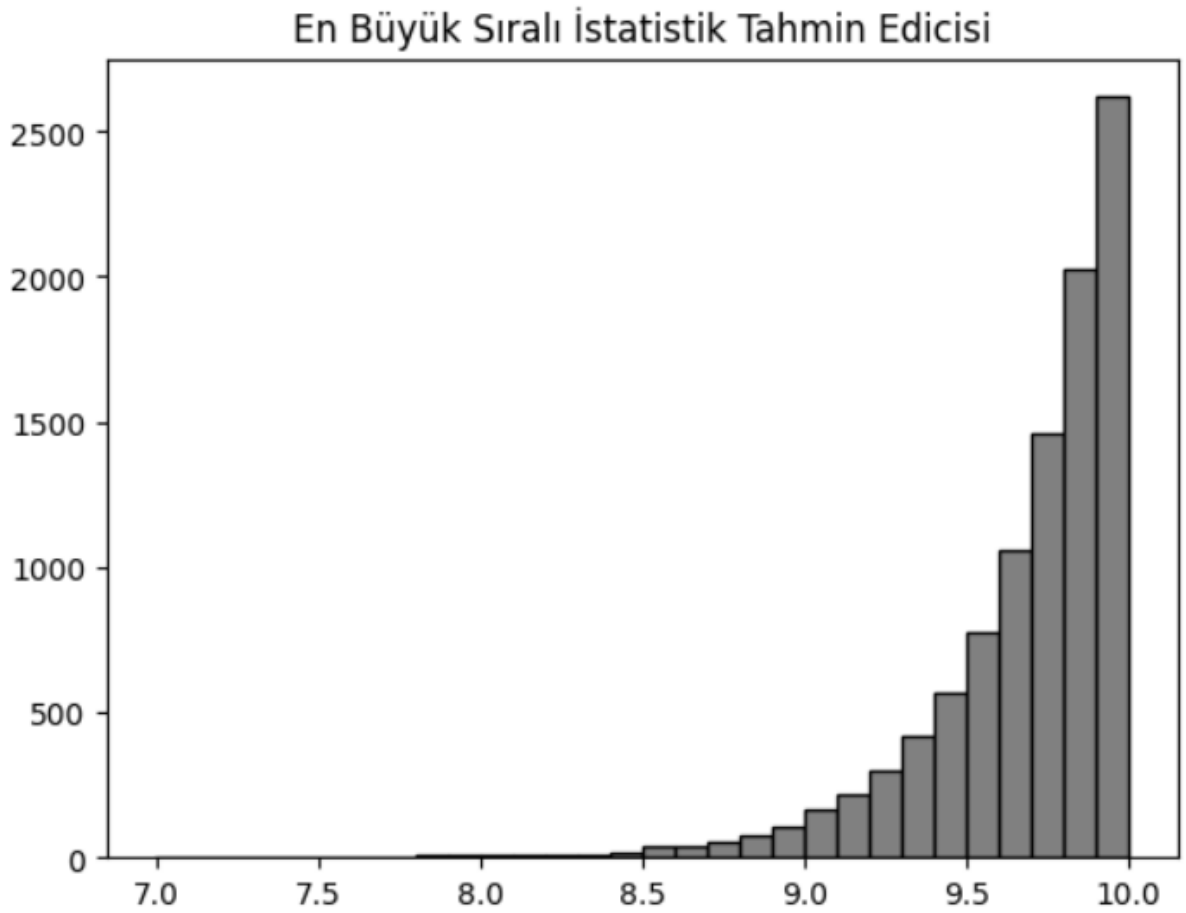
Yorum ve Deęerlendirme

Tahmin edicinin ortalamalarının daęılımının ortalama deęeri : 9.684062344483591

Tahmin edicinin varyanslarının daęılımının ortalama deęeri : 0.0956416150728507

En büyük sıralı istatistik olan hesaplanan , test edilen istatistik daęılımı olan uniform daęılım için tutarlı ve yansız olduęundan dolayı iyi bir tahmin edicidir diyebiliriz.

Örneklem büyüklüęü arttıkça tahmin deęerinin parametre olan θ yakınsadıęını görüyoruz bu özellięe ise tutarlılık özellięi denir. Aşaęıda verilen histogram grafięinden de bu çıkarıma ulaşabiliriz:



Kodlar ve Açıklamaları

ÖDEV 1

```
import numpy as np
import matplotlib.pyplot as plt # Daha sonra kullanma için

rng = np.random.default_rng(seed=26)
kitle = rng.normal(loc=50, scale=10, size=10000)
print(kitle)
```

```
[30.74935502 19.81204769 61.54463147 ... 60.36609293 49.42113287
41.62076388]
```

Python üzerinden matematiksel ve istatistiksel değerleri/ölçümleri hesaplamak için numpy kütüphanesi ve histogram çizimleri için matplotlib kütüphanesinin .pyplot modülünü import ile çağırıyoruz.

Normal dağılıma sahip kitlemizi üretmek için numpy üzerinden np.random.normal() fonksiyonunu kullanabiliriz ancak oluşan kitle değerlerimizi kod her çalıştırdığında yeniden oluşturması gibi bir durumla karşı karşıya kalıyoruz bu durumu seed() mantığı ile çözebiliriz bu yöntemi kullanarak rng ile belirli bir seed ayarlıyoruz ve bu seed ile kodumuzu her çalıştırdığımızda aynı random yöntemi (seed) ile çektiğinden dolayı değerler değişmiyor. Bu sayede kitlemizi oluşturmuş ve “kitle” değişkeninde saklıyoruz.

Bölüm 1 – Kitle Parametrelerinin Hesaplanması

```
kitle_ort = np.mean(kitle)
print(f"Kitle Ortalaması = {kitle_ort}")
```

```
Kitle Ortalaması = 50.10651655016294
```

Kitle ortalamasını np.mean() modülü ile hesaplıyoruz. Ve “kitle_ort” değişkeninde tutuyoruz.

```
var_kitle = np.var(kitle)
print(f"Kitle Varyansı = {var_kitle}")
```

```
Kitle Varyansı = 99.12312792831453
```

Bölüm 2 – Farklı Örneklem Büyüklükleri ile Nokta Tahmini

```
sample1 = rng.choice(kitle, size=10)
sample2 = rng.choice(kitle, size=30)
sample3 = rng.choice(kitle, size=100)
sample4 = rng.choice(kitle, size=500)
```

Oluşturduğumuz rng seed’i ile .choice() modülü sayesinde farklı büyüklüklerde örneklemelerimizi çekiyoruz ev bunlar farklı değişkenlere atıyoruz.

```
ort_sample1 = np.mean(sample1)
print(f"1.örneklem için X ortalama = {ort_sample1}")
ort_sample2 = np.mean(sample2)
print(f"2.örneklem için X ortalama = {ort_sample2}")
ort_sample3 = np.mean(sample3)
print(f"3.örneklem için X ortalama = {ort_sample3}")
ort_sample4 = np.mean(sample4)
print(f"4.örneklem için X ortalama = {ort_sample4}")
```

```
1.örneklem için X ortalama = 51.23863490214186
2.örneklem için X ortalama = 49.768142057628886
3.örneklem için X ortalama = 50.045342377970684
4.örneklem için X ortalama = 50.80455027187787
```

Oluşturduğumuz örneklem için ortalamalarını hesaplıyoruz.

```
var_sample1 = np.var(sample1, ddof=1)
print(f"1.örneklem için örneklem varyansı = {var_sample1}")
var_sample2 = np.var(sample2, ddof=1)
print(f"2.örneklem için örneklem varyansı = {var_sample2}")
var_sample3 = np.var(sample3, ddof=1)
print(f"3.örneklem için örneklem varyansı = {var_sample3}")
var_sample4 = np.var(sample4, ddof=1)
print(f"4.örneklem için örneklem varyansı = {var_sample4}")
```

```
1.örneklem için örneklem varyansı = 65.14485914208004
2.örneklem için örneklem varyansı = 48.99186399951632
3.örneklem için örneklem varyansı = 91.00381011345227
4.örneklem için örneklem varyansı = 105.47933129173595
```

Oluşturulan örneklem için varyans değerleri hesaplanmıştır. Örneklem varyansı hesaplarken serbestlik derecesini .var modülü ddof= parametresi ile belirliyor ve örneklem büyüklüğü olan n'den çıkarılacak değeri girebilmesini sağlıyor. Burada ddof=1 değeri ile örneklem varyansının paydasında n-1 değeri belirlenmiştir.

Bölüm 3 – Yansızlık ve Tutarlılık Simülasyonu

```
n10_means = []
n30_means = []
n100_means = []
n500_means = []

for i in range(1000):
    sample10 = rng.choice(kitle,size=10)
    n10_means.append(np.mean(sample10))

    sample30 = rng.choice(kitle,size=30)
    n30_means.append(np.mean(sample30))

    sample100 = rng.choice(kitle,size=100)
    n100_means.append(np.mean(sample100))

    sample500 = rng.choice(kitle,size=500)
    n500_means.append(np.mean(sample500))
```

```
print(f"n = 10 için örneklem ortalamalarının ortalaması : {np.mean(n10_means)}, Varyans
print(f"n = 30 için örneklem ortalamalarının ortalaması : {np.mean(n30_means)}, Varyans
print(f"n = 100 için örneklem ortalamalarının ortalaması : {np.mean(n100_means)}, Varya
print(f"n = 500 için örneklem ortalamalarının ortalaması : {np.mean(n500_means)}, Varya
```

```
n = 10 için örneklem ortalamalarının ortalaması : 50.033594742042666, Varyansı : 9.9834
6703401207
n = 30 için örneklem ortalamalarının ortalaması : 50.077417967920056, Varyansı : 3.3405
473241528965
n = 100 için örneklem ortalamalarının ortalaması : 50.11983083020773, Varyansı : 0.9733
018129139838
n = 500 için örneklem ortalamalarının ortalaması : 50.099973242594444, Varyansı : 0.186
87106718520716
```

Farklı örneklem büyüklükleri ile ortalama değerleri hesaplanıp değişkenlere atanmıştır.

Daha sonra bu örneklem ortalamalarını oluşturduğu örneklemelerin ortalamaları alınmış ve örnek ortalamasının ne kadar iyi tahmin edici olup olmadığı incelenmiştir.

```
plt.hist(n10_means, bins=30, color="red", edgecolor="black")
plt.title("n=10 İçin Örneklem Ortalamaları Dağılımı")
plt.show()

plt.hist(n30_means, bins=30, color="yellow", edgecolor="black")
plt.title("n=30 İçin Örneklem Ortalamaları Dağılımı")
plt.show()

plt.hist(n100_means, bins=30, color="cyan", edgecolor="black")
plt.title("n=100 İçin Örneklem Ortalamaları Dağılımı")
plt.show()

plt.hist(n500_means, bins=30, color="olive", edgecolor="black")
plt.title("n=500 İçin Örneklem Ortalamaları Dağılımı")
plt.show()
```

Histogram grafikleri matplotlib kütüphanesini .pyplot modülünün .hist fonksiyonu ile çizilmiştir. .title fonksiyonu ile bu histogram grafikleri adlandırılmış daha sonra .show fonksiyonu ile gösterilmiştir.

ÖDEV 2

```
rng2 = np.random.default_rng(seed=26)
```

rng2 ile seed() belirleyip değerlerin değişmesi önlenmiştir.

```
t1_mu = []
t2_mu = []
t3_mu = []

for i in range(1000):
    sample = rng2.normal(loc=20, scale=15, size=30)

    t1 = np.mean(sample)
    t1_mu.append(t1)

    t2 = (sample[0]+sample[1])/2
    t2_mu.append(t2)

    t3 = (sample[0]-(2*sample[1])+(3*sample[2]))/6
    t3_mu.append(t3)
```

3 tahmin edici içinde 1000 farklı örneklem oluşturulmuş ve tahmin edici değerleri değişkenlere atanmıştır. .append() fonksiyonu ile dizin işlemleri yapılırken sondan ekleme ile atama işlemleri yapılmıştır.

```

t1_sigma_sq = []
t2_sigma_sq = []

for i in range(1000):
    sample = rng2.normal(loc=20,scale=15,size=30)

    t1 = np.var(sample,ddof=1)
    t1_sigma_sq.append(t1)

    t2 = ((sample[0]-sample[1])**2)/2
    t2_sigma_sq.append(t2)

```

2 tahmin edici için de 1000 farklı örneklem çekilmiş ve hesaplanan tahmin edici değerleri değişkenlere kaydedilmiştir.

Bölüm 2 – Tahmin Ediciler Performansı

```

# Yansızlık hesabı
"""
mü için ise t1 ve t2 yansız ancak t3 yanlış dır deriz aldığımız o t.e lere ait
ortalamalar ortalaması bizi bu sonuca götürdü (beklenen değer parametreye eşit
olmadığı görüldü)

varyans için se test ettiğimiz t1 ve t2 t.e (tahmin edicileri) varyanslarına ait
örneklemimizin ortalamasını alarak nereye götrüdüğüne baktık ve ikisi de bizi
dağılımımızın varyansına götürdü bu iki t.e ye de yansız diyebildik.
"""

print(np.mean(t1_mu)-20)
print(np.mean(t2_mu)-20)
print(np.mean(t3_mu)-20)

print(np.mean(t1_sigma_sq)-20)
print(np.mean(t2_sigma_sq)-20)

```

```

-0.001714921444847306
0.342679087543754
-14.205737200259803
203.46722565597062
206.87207521551426

```

Tahmin edicilerin ortalamaları ve yansızlık değerleri için yan değerleri hesaplandı.

```
# varyansı

t1_mu_var = np.var(t1_mu,ddof=1)
t2_mu_var = np.var(t2_mu,ddof=1)
t3_mu_var = np.var(t3_mu,ddof=1)

print(f"mu T1 t.e dağılımının varyansı = {t1_mu_var}")
print(f"mu T2 t.e dağılımının varyansı = {t2_mu_var}")
print(f"mu T3 t.e dağılımının varyansı = {t3_mu_var}")

t1_sigma_sq_var = np.var(t1_sigma_sq,ddof=1)
t2_sigma_sq_var = np.var(t2_sigma_sq,ddof=1)

print(f"sigma kare T1 t.e dağılımının varyansı = {t1_sigma_sq_var}")
print(f"sigma kare T2 t.e dağılımının varyansı = {t2_sigma_sq_var}")

mu T1 t.e dağılımının varyansı = 7.309909836875564
mu T2 t.e dağılımının varyansı = 112.96634767182057
mu T3 t.e dağılımının varyansı = 95.33227380212516
sigma kare T1 t.e dağılımının varyansı = 3324.480145242315
sigma kare T2 t.e dağılımının varyansı = 101224.10628780416
```

Tahmin edicilerin varyans değerleri hesaplandı ve değişkenlere atandı.

```
# Hata kareler ortalaması

t1_mu_yan = np.mean(t1_mu)-20
t2_mu_yan = np.mean(t2_mu)-20
t3_mu_yan = np.mean(t3_mu)-20

hko_t1_mu = t1_mu_var + t1_mu_yan**2
hko_t2_mu = t2_mu_var + t2_mu_yan**2
hko_t3_mu = t3_mu_var + t3_mu_yan**2

print(f" T1_mu t.e için HKO = {hko_t1_mu}")
print(f" T2_mu t.e için HKO = {hko_t2_mu}")
print(f" T3_mu t.e için HKO = {hko_t3_mu}")

t1_sigma_sq_yan = t1_sigma_sq_var-225
t2_sigma_sq_yan = t2_sigma_sq_var-225

hko_t1_sigma_sq = t1_sigma_sq_var + t1_sigma_sq_yan**2
hko_t2_sigma_sq = t2_sigma_sq_var + t2_sigma_sq_yan**2

print(f" T1_var t.e için HKO = {hko_t1_sigma_sq}")
print(f" T2_var t.e için HKO = {hko_t2_sigma_sq}")

T1_mu t.e için HKO = 7.3099127778311255
T2_mu t.e için HKO = 113.08377662886039
T3_mu t.e için HKO = 297.1352432049704
T1_var t.e için HKO = 9610101.650896564
T2_var t.e için HKO = 10200920695.041449
```

Tahmin edicilerin Hata Kareler Ortalaması (HKO) hesaplandı.

Bölüm 3 – Etkinlik

```
print(f"T2_mu'nin T1_mu'e etkinliği = {t1_mu_var/t2_mu_var}")
```

T2_mu'nin T1_mu'e etkinliği = 0.0647087383767743

[128]:

```
print(f"T2_mu'nin T1_mu'e etkinliği = {t1_sigma_sq_var/t2_sigma_sq_var}")
```

T2_mu'nin T1_mu'e etkinliği = 0.032842771027190194

Etkinlik değerleri formül gereği varyansların oranı şeklinde hesaplanmış ve varyansı küçük olan paya büyük olan paydaya yazılacak şekilde hesaplanmıştır ki değer 0 ve 1 arasında çıksın gösterim açısından bunu tercih edilmiştir.

ÖDEV 3

Bölüm 1 - Simülasyon

```
Xnler = []
for i in range(10000):
    orneklem = np.random.uniform(low=0, high=10, size=30)
    Xnler.append(np.max(orneklem))
print(np.array(Xnler))
```

[9.73586936 9.24083776 9.78039627 ... 9.83991173 9.98925869 9.97336416]

En büyük sıralı istatistik için 10.000 örneklem üretilip bu örneklemelerin en büyük sıralı istatistiği alınarak “Xnler” değişkenine atanmıştır.

Bölüm 2 – Tahmin Edicinin Davranışı

```
Xnler_ort = np.mean(Xnler)
Xnler_var = np.var(Xnler)

print(f"Teta_sapka (Xn -En buyuk sıralı ist-) t.e'nin ortalaması = {Xnler_ort}")
print(f"Teta_sapka (Xn -En buyuk sıralı ist-) t.e'nin varyansı = {Xnler_var}")
```

Teta_sapka (Xn -En buyuk sıralı ist-) t.e'nin ortalaması = 9.684062344483591

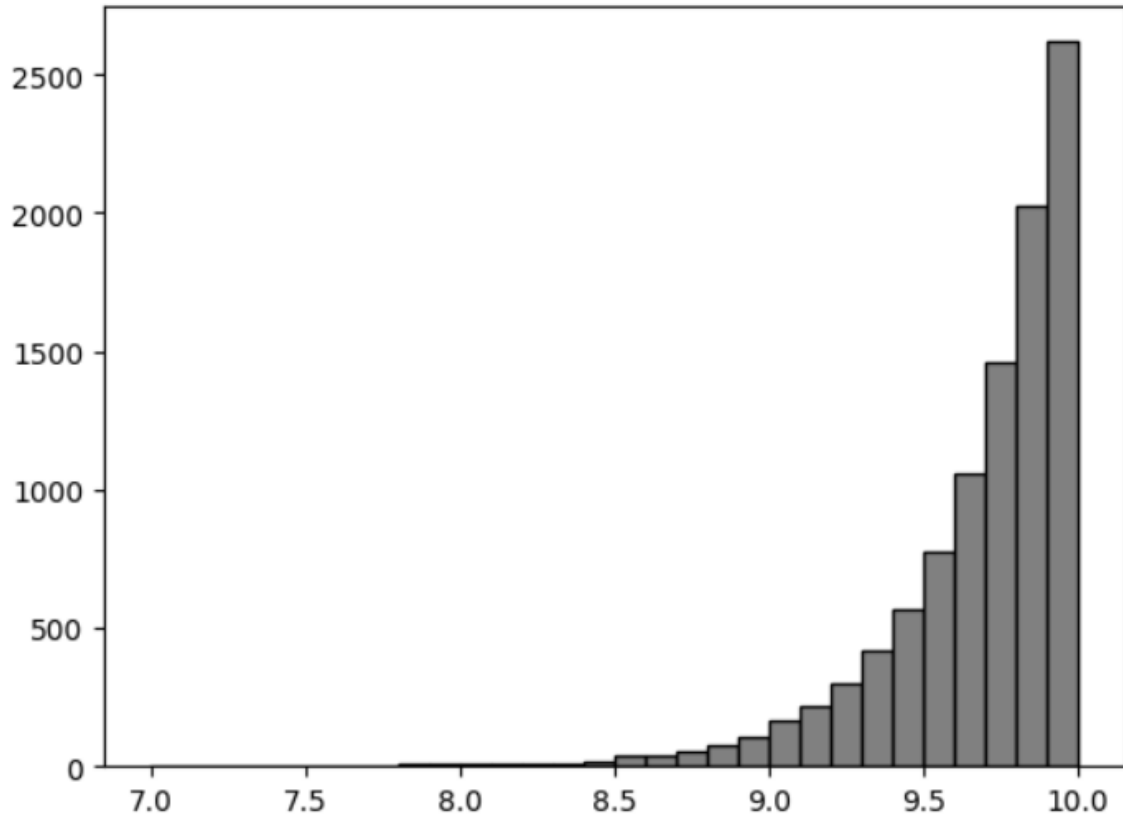
Teta_sapka (Xn -En buyuk sıralı ist-) t.e'nin varyansı = 0.0956416150728507

Tahmin edicinin davranışını görebilmek için oluşturulmuş olan 10.000 örneğin en büyük sıralı istatistiklerinden oluşan yeni dağılımın ortalaması ve varyansı alınmıştır.

```
plt.hist(Xnler, bins=30, color="grey", edgecolor="black")
plt.title("En Büyük Sıralı İstatistik Tahmin Edicisi")
plt.show
```

.hist modülü ile histogram grafiği oluşturulmuş, bins parametresi ile kaç kutucuğa bölüneceği belirlenmiş, color parametresi ile kutuların renkleri belirlenmiş, edgecolor parametresi ile kutuların sınırlarının renkleri belirlenmiş ve görsellik bu şekilde düzenlenmiştir. .title modülü ile histogram grafiğinin başlığı belirlenmiş, .show modülü ile gösterilmiştir.

En Büyük Sıralı İstatistik Tahmin Edicisi



KAYNAKÇA :

1. <https://numpy.org/doc/2.4/user/> (ÖDEV1)(ÖDEV2)(ÖDEV3) (Tüm ödevler için numpy'ın dokümantasyon sayfasından yardım alarak parametrelere ve ihtiyacım olan modul ve fonksiyonlara baktım. Bu ödev için tüm numpy işlemleri için de yeterli oldu)
2. <https://docs.python.org/3/tutorial/> (ÖDEV1)(ÖDEV2)(ÖDEV3)(Array ve for döngüleri için kullandım)
3. <https://www.kilicaslan.nom.tr/latex/> (Word'de ve Jupyter'de markdown için LaTeX formatında denklem yazımına baktım.)
4. <https://numpy.org/doc/stable/reference/random/generated/numpy.random.normal> (ÖDEV-1 Bölüm-1)
5. <https://stackoverflow.com/questions/32172054/how-can-i-retrieve-the-current-seed-of-numpys-random-number-generator> (ÖDEV 1)(Seed mantığı için örnek görmek istedim çok yardımcı olmadı ama seed mantığını buradan anlamaya çalışmışım)
6. Dr. Haydar DEMİRHAN & Dr. Canan HAMURKAROĞLU (2016) *İstatiksel Yöntemlere Giriş* (3.Baskı) (ÖDEV1-2-3 Teorik bilgiyi buradan aldım, tutarlılık, yansızlık vb.)